ELEC 475 Lab 2
Post Lab Report

Zachary Gerhard | Student number 20359351
Finn Corkery | Student number 20341468
October 22 2026

Study overview

We compared two LeNet variants, three normalization variants of AlexNet for 64 × 64 inputs, and torchvision AlexNet on CIFAR-10 and Tiny ImageNet-200. BatchNorm gave the strongest mean accuracy among the 10-epoch normalization runs. The best tested configuration used BatchNorm, dropout, augmentation and a learning rate of 0.0005: its final validation accuracy was 82.79 ± 0.61% on CIFAR-10 and 35.62 ± 0.59% on Tiny ImageNet.

The matched augmentation experiment reduced the clean training-validation gap on both datasets. Tiny ImageNet remained substantially harder, and two individual runs stayed at chance accuracy. We retained those runs when calculating the reported means and standard deviations.

Experiment settings

Setting	Value
Execution	Ubuntu under WSL; NVIDIA GeForce RTX 3050 Laptop GPU
Recorded runtime	Python 3.14.7; PyTorch 2.14.1+cu130; CUDA
Optimizer and loss	Adam; weight decay 0.0001; cross entropy on logits
Batch and loading	Batch size 128; 2 workers; shuffled training; drop_last=True
Random seeds	42, 123 and 999 for every configuration and dataset
Training budgets	LeNets: 5 epochs; normalization and Zoo: 10; augmentation and custom LR: 15
Learning rate	0.001, except the custom augmented trial at 0.0005
Initialization	Random initialization throughout; no pretrained weights; no early stopping

All 54 runs completed their requested budgets. CIFAR-10 used 50,000 training images and its official 10,000-image test split as validation, following the course starter. Tiny ImageNet used its separate 100,000-image training and 10,000-image annotated validation splits, with one consistent class mapping. Both datasets supplied RGB tensors of shape 3 × 64 × 64.

Metric definitions

$$F_{1,c} = \frac{2TP_c}{2TP_c + FP_c + FN_c}, \quad F_{1,\text{macro}} = \frac{1}{K}\sum_{c=1}^{K} F_{1,c}, \quad \Delta = A_{\text{train, clean}} - A_{\text{val}}$$

Accuracy is top-1 accuracy. F1 is computed per class and averaged across all K classes; an undefined class F1 contributes zero. Percentages and percentage-point gaps are reported as mean ± sample SD across the three seeds. Tables use the final epoch, rather than the best epoch. Clean training accuracy evaluates the final model in eval mode without random augmentation; online training accuracy in the curves includes dropout and changing weights.

1 LeNet architectures and receptive fields

For the layer tables, k/s/p denotes kernel size, stride and padding, and output dimensions are channels × height × width. All convolutions and linear layers include a bias. K = 10 for CIFAR-10 and K = 200 for Tiny ImageNet. The output is a vector of K logits.

$$O = \left\lfloor \frac{I + 2p - k}{s} \right\rfloor + 1, \quad r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}, \quad j_\ell = s_\ell j_{\ell-1}$$

$$P_{\text{conv}} = C_{\text{out}}(C_{\text{in}} k^2 + 1), \quad P_{\text{linear}} = N_{\text{out}}(N_{\text{in}} + 1), \quad P_{\text{BN}} = 2C$$

The receptive field r and sampling jump j start at 1. These formulas use dilation 1. Padding changes the output size; theoretical receptive fields can include padded positions. Dense layers combine all positions in the final feature grid.

Table 1 Base LeNet layer dimensions and trainable parameters

Layer	k/s/p	Output	Parameters	r / j
Conv1 + Tanh	5/1/0	6 × 60 × 60	456	5 / 1
Average pool	2/2/0	6 × 30 × 30	0	6 / 2
Conv2 + Tanh	5/1/0	16 × 26 × 26	2,416	14 / 2
Average pool	2/2/0	16 × 13 × 13	0	16 / 4
Flatten → FC1 + Tanh	-	2704 → 120	324,600	Whole grid
FC2 + Tanh	-	120 → 84	10,164	Whole grid
FC3 logits	-	84 → K	85K	Whole grid

Table 2 Modern LeNet layer dimensions and trainable parameters

Layer	k/s/p	Output	Parameters	r / j
Conv1 + ReLU	5/1/2	6 × 64 × 64	456	5 / 1
Overlapping max pool	3/2/0	6 × 31 × 31	0	7 / 2
Conv2 + ReLU	5/1/2	16 × 31 × 31	2,416	15 / 2
Overlapping max pool	3/2/0	16 × 15 × 15	0	19 / 4
Flatten → FC1 + ReLU	-	3600 → 120	432,120	Whole grid
FC2 + ReLU	-	120 → 84	10,164	Whole grid
FC3 logits	-	84 → K	85K	Whole grid

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \qquad \text{ReLU}(x) = \max(0, x)$$

Tanh saturates for large positive or negative inputs, where its derivative approaches zero. ReLU retains a unit derivative for positive inputs. The modern network also preserves larger spatial maps with padding and uses overlapping 3 × 3 max pooling. These changes are bundled, so this experiment does not isolate the effect of activation alone. The larger 3600-element flatten layer also increases the parameter count.

After five epochs, Modern LeNet exceeded Base LeNet by 12.89 accuracy points on CIFAR-10 and 8.81 points on Tiny ImageNet. It also outperformed the 10-epoch Zoo baseline on both datasets, showing that a larger parameter count alone did not guarantee stronger performance in these runs.

2 AlexNet for 64 by 64 images

AlexNet-64 uses five convolutional layers, three max-pooling layers and three linear layers. Every convolution is followed by ReLU. BatchNorm, when selected, precedes ReLU in all five stages. LRN, when selected, follows ReLU in the first two stages only. Dropout with p = 0.5 precedes each of the first two linear layers.

Table 3 AlexNet-64 dimensions and parameters before normalization

Layer	k/s/p	Output	Parameters	r / j
Conv1	5/2/2	64 × 32 × 32	4,864	5 / 2
Max pool 1	3/2/0	64 × 15 × 15	0	9 / 4
Conv2	5/1/2	192 × 15 × 15	307,392	25 / 4
Max pool 2	3/2/0	192 × 7 × 7	0	33 / 8
Conv3	3/1/1	384 × 7 × 7	663,936	49 / 8
Conv4	3/1/1	384 × 7 × 7	1,327,488	65 / 8
Conv5	3/1/1	256 × 7 × 7	884,992	81 / 8
Max pool 3	3/2/0	256 × 3 × 3	0	97 / 16
Flatten → FC1 + ReLU	-	2304 → 1024	2,360,320	Whole grid
FC2 + ReLU	-	1024 → 1024	1,049,600	Whole grid
FC3 logits	-	1024 → K	1025K	Whole grid

For example, Conv3 follows a receptive field of 33 and a jump of 8, so its field is 33 + (3 − 1) × 8 = 49 pixels. The final theoretical field of 97 exceeds the 64-pixel image width because it includes padding.

Normalization and regularization

$$\widehat{x}_{nchw} = \frac{x_{nchw} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}, \qquad y_{nchw} = \gamma_c \widehat{x}_{nchw} + \beta_c$$

BatchNorm estimates the mean and variance for each channel across batch and spatial positions during training. Learned scale γ and shift β preserve flexibility. Evaluation uses running statistics. The five BN layers add 2 × (64 + 192 + 384 + 384 + 256) = 2,560 trainable parameters; running means and variances are buffers.

$$b_i = \frac{a_i}{\left(k_0 + \frac{\alpha}{n} \sum_{j \in \mathcal{N}(i)} a_j^2\right)^{\beta}}, \quad n = 5,\ \alpha = 10^{-4},\ \beta = 0.75,\ k_0 = 2$$

LRN divides each activation by a response formed from a neighborhood of five channels at the same spatial position. The expression includes PyTorch's α/n convention. LRN has no learned parameters or running statistics. Dropout randomly zeros half the selected dense activations during training, with scaling to preserve their expectation; it is disabled during evaluation.

3 Torchvision AlexNet and parameter totals

The Zoo baseline uses torchvision AlexNet with weights=None and replaces its last linear layer with K outputs. Its original 11 × 11 stride-4 first convolution downsamples much more rapidly than AlexNet-64. At 64 × 64 input, the last max pool produces a 1 × 1 feature map. Adaptive average pooling expands that grid to 6 × 6 for the original classifier, without recovering spatial detail.

Table 4 Zoo AlexNet dimensions and trainable parameters

Layer	k/s/p	Output	Parameters	r / j
Conv1 + ReLU	11/4/2	64 × 15 × 15	23,296	11 / 4
Max pool 1	3/2/0	64 × 7 × 7	0	19 / 8
Conv2 + ReLU	5/1/2	192 × 7 × 7	307,392	51 / 8
Max pool 2	3/2/0	192 × 3 × 3	0	67 / 16
Conv3 + ReLU	3/1/1	384 × 3 × 3	663,936	99 / 16
Conv4 + ReLU	3/1/1	256 × 3 × 3	884,992	131 / 16
Conv5 + ReLU	3/1/1	256 × 3 × 3	590,080	163 / 16
Max pool 3	3/2/0	256 × 1 × 1	0	195 / 32
Adaptive average pool	to 6 × 6	256 × 6 × 6	0	Same input support
Flatten → FC1 + ReLU	-	9216 → 4096	37,752,832	Whole grid
FC2 + ReLU	-	4096 → 4096	16,781,312	Whole grid
FC3 logits	-	4096 → K	4097K	Whole grid

The Zoo classifier uses dropout p = 0.5 before FC1 and FC2. ReLU, pooling, dropout and LRN have no trainable parameters.

Table 5 Parameter breakdowns for the implemented networks

Model	Conv	Dense K=10	BN	Total K=10	Total K=200
Base LeNet	2,872	335,614	0	338,486	354,636
Modern LeNet	2,872	443,134	0	446,006	462,156
AlexNet none / LRN	3,188,672	3,420,170	0	6,608,842	6,803,592
AlexNet BN	3,188,672	3,420,170	2,560	6,611,402	6,806,152
Zoo AlexNet	2,469,696	54,575,114	0	57,044,810	57,823,240

For K = 10, the dense classifier contains about 95.7% of the Zoo model's parameters, compared with about 51.7% for AlexNet-64 without normalization. Increasing K changes only the output layer. We use exact layer counts throughout: the specified AlexNet-64 layers give 6.609M parameters without BN, rather than the handout's approximate 6.3M.

4 Computational complexity and measured runtime

Table 7.3a combines architecture settings with measurements on the RTX 3050 Laptop GPU. In each paired numeric cell, the first line is CIFAR-10 and the second is Tiny ImageNet. Epoch time is the mean of the three run-level averages. It includes training and validation, and excludes the final clean-training evaluation, profiling and file output.

Table 7.3a Architecture configuration and computational complexity

Model	Conv layers	Normalization and regularization	Params M	FLOPs M/image	Epoch s	Latency ms/image
Base LeNet	2	Tanh; avg pool No augmentation	0.3385 0.3546	7.17 7.20	12.40 24.61	0.4557 0.4038
Modern LeNet	2	ReLU; max pool No augmentation	0.4460 0.4622	9.19 9.22	12.11 24.97	0.5250 0.4368
AlexNet none	5	None; dropout 0.5 No augmentation	6.6088 6.8036	436.70 437.08	23.44 53.14	1.1260 1.1585
AlexNet LRN	5	LRN; dropout 0.5 No augmentation	6.6088 6.8036	436.70 437.08	33.68 70.31	1.4956 1.4853
AlexNet BN	5	BN; dropout 0.5 No augmentation	6.6114 6.8062	437.97 438.36	26.51 49.29	1.1575 1.1792
AlexNet BN + aug	5	BN; dropout 0.5 Augmentation	6.6114 6.8062	437.97 438.36	28.37 58.88	1.1855 1.3020
Zoo AlexNet	5	Adaptive pool; dropout 0.5 No augmentation	57.0448 57.8232	188.20 189.76	29.01 48.44	2.2456 2.2994

Budgets: 5 epochs for LeNets; 10 for AlexNet none/LRN/BN and Zoo; 15 for BN + augmentation. Augmentation applies only during training. Adam weight decay 0.0001 applies to all runs.

Profiling convention

THOP estimates forward MACs on one 3 × 64 × 64 image; FLOPs equal 2 × MACs. Some operators, including LRN, are uncounted. Parameters were counted directly, and a deep copy was profiled to protect BatchNorm state.

Latency is the median of 50 synchronized eval-mode forwards after 10 warmup forwards, with batch size 1 and input already on CUDA. Decoding, augmentation and host-to-device transfer are excluded. Section 11 reproduces the terminal tables.

Computational tradeoffs

CIFAR-10 LRN took 1.4956 ms versus 1.1260 ms without normalization, about 33% longer despite identical estimated FLOPs. The estimate omits LRN work. BatchNorm took 1.1575 ms, about 3% longer than no normalization. BN with and without augmentation has the same inference graph; separate timing runs produced slightly different measurements.

Zoo AlexNet has fewer estimated FLOPs because of rapid early downsampling, but its large classifier accompanies the slowest CIFAR-10 latency, 2.2456 ms. FLOPs alone did not predict latency. Modern LeNet was much smaller and faster than the AlexNet variants, making it a useful low-cost baseline.

Training time varied across runs under unrecorded system conditions. For example, CIFAR-10 BN without augmentation at 15 epochs averaged 25.50, 26.74 and 41.49 s per epoch across the three seeds. Consequently, the lower aggregate epoch time for the augmented row cannot establish that augmentation is faster.

5 Predictive performance and extended ablations

Table 7.3b Final validation performance as mean ± sample SD in percent n = 3

Model	CIFAR-10 Accuracy	CIFAR-10 Macro F1	Tiny ImageNet Accuracy	Tiny ImageNet Macro F1
Base LeNet	52.71 ± 0.79	52.30 ± 0.62	9.99 ± 0.25	7.86 ± 0.34
Modern LeNet	65.61 ± 1.44	65.52 ± 1.38	18.80 ± 0.42	16.63 ± 0.57
AlexNet none	72.73 ± 1.56	72.89 ± 1.36	15.26 ± 1.84	12.56 ± 1.99
AlexNet LRN	74.26 ± 1.34	74.20 ± 1.50	10.47 ± 8.64	8.37 ± 7.24
AlexNet BN	77.79 ± 3.16	77.81 ± 3.38	29.05 ± 1.18	27.41 ± 1.12
AlexNet BN + aug	81.98 ± 0.86	81.89 ± 0.79	33.53 ± 2.17	32.18 ± 2.32
Zoo AlexNet	64.85 ± 1.32	64.86 ± 1.11	8.29 ± 6.77	6.31 ± 5.53

Seeds are 42, 123 and 999. Epoch budgets are stated in Section 4. Cross-row differences in this master table combine architecture and budget differences; the following comparisons use matched 15-epoch budgets.

Table 6a Extended ablation on CIFAR-10 15 epochs mean ± sample SD

BN configuration	Clean train Accuracy %	Validation Accuracy %	Validation Macro F1 %	Gap Δ pp
No aug LR 0.001	90.82 ± 0.64	80.55 ± 0.63	80.50 ± 0.68	10.27 ± 0.13
Aug LR 0.001	86.02 ± 1.39	81.98 ± 0.86	81.89 ± 0.79	4.04 ± 0.53
Aug LR 0.0005	87.60 ± 0.72	82.79 ± 0.61	82.58 ± 0.42	4.81 ± 0.12

Mean epoch seconds, in row order: 31.24, 28.37, 32.74. Every row has 6.611402M parameters and an estimated 437.966M FLOPs per image.

Table 6b Extended ablation on Tiny ImageNet 15 epochs mean ± sample SD

BN configuration	Clean train Accuracy %	Validation Accuracy %	Validation Macro F1 %	Gap Δ pp
No aug LR 0.001	44.05 ± 1.27	33.23 ± 1.00	31.96 ± 1.45	10.82 ± 0.52
Aug LR 0.001	38.17 ± 1.98	33.53 ± 2.17	32.18 ± 2.32	4.64 ± 0.35
Aug LR 0.0005	41.64 ± 1.41	35.62 ± 0.59	34.26 ± 0.58	6.01 ± 0.83

Mean epoch seconds, in row order: 47.76, 58.88, 56.27. Every row has 6.806152M parameters and an estimated 438.355M FLOPs per image. The custom ablation changes only the learning rate relative to the augmented reference.

$$\bar{x} = \frac{1}{3}\sum_{i=1}^{3} x_i, \qquad s = \sqrt{\frac{\sum_{i=1}^{3}(x_i - \bar{x})^2}{3-1}}$$

We computed each run’s gap before aggregating, so its SD includes the paired variation in clean training and validation accuracy. With only three seeds, these SDs describe the observed variation; we did not perform a statistical significance test.

6 Normalization results and learning curves

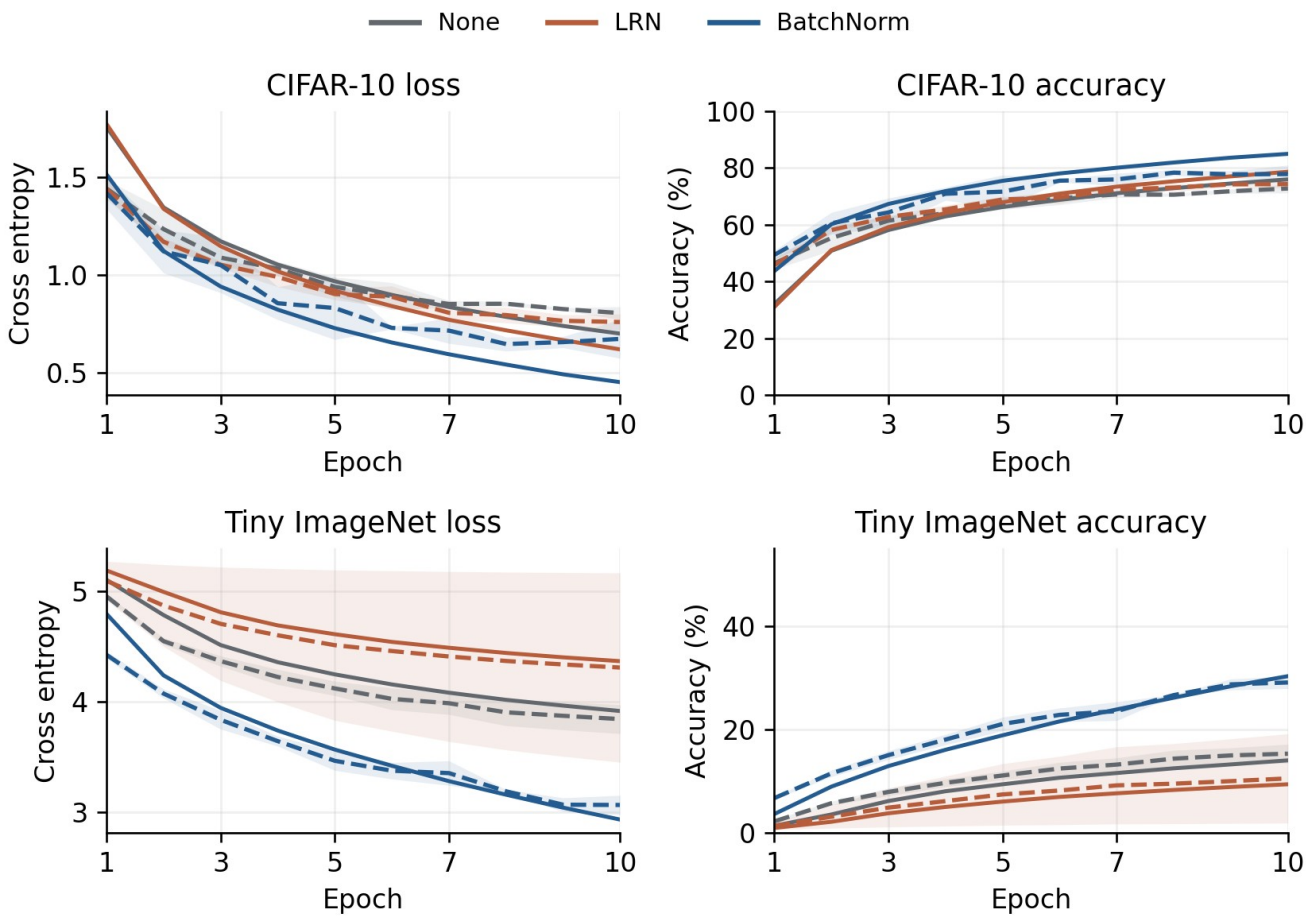


Figure 1 Normalization over 10 epochs. Solid lines show online training and dashed lines show validation. Lines are the means of three seeds; shading is ±1 sample SD of validation. The upper row is CIFAR-10 and the lower row is Tiny ImageNet.

BatchNorm had the best mean final accuracy and Macro F1 among the three normalization choices. On CIFAR-10 its 77.79% mean accuracy exceeded no normalization by 5.06 points and LRN by 3.53 points. The answer to whether BN improved the average validation metrics is therefore yes. CIFAR-10 BN still had substantial final-epoch variation (SD 3.16 points), so the result does not show that BN reduces variation in every setting.

On Tiny ImageNet, BN achieved 29.05 ± 1.18% accuracy, compared with 15.26 ± 1.84% without normalization and 10.47 ± 8.64% with LRN. The LRN mean includes a seed-42 run at chance throughout training. BN's improvement is especially clear here, although the experiment does not directly measure gradients or activation distributions.

BN rescales each channel before ReLU, making the scale presented to later layers less sensitive to earlier weight changes while retaining learned scale and shift. Ioffe and Szegedy motivated BN through internal covariate shift [4]. Later work connected its optimization benefits to a smoother loss landscape and more predictable gradients [5]. Our curves are consistent with easier optimization under BN, but they do not establish a particular mechanism. LRN only rescales neighboring channel responses and did not provide the same improvement.

7 Augmentation and the generalization gap

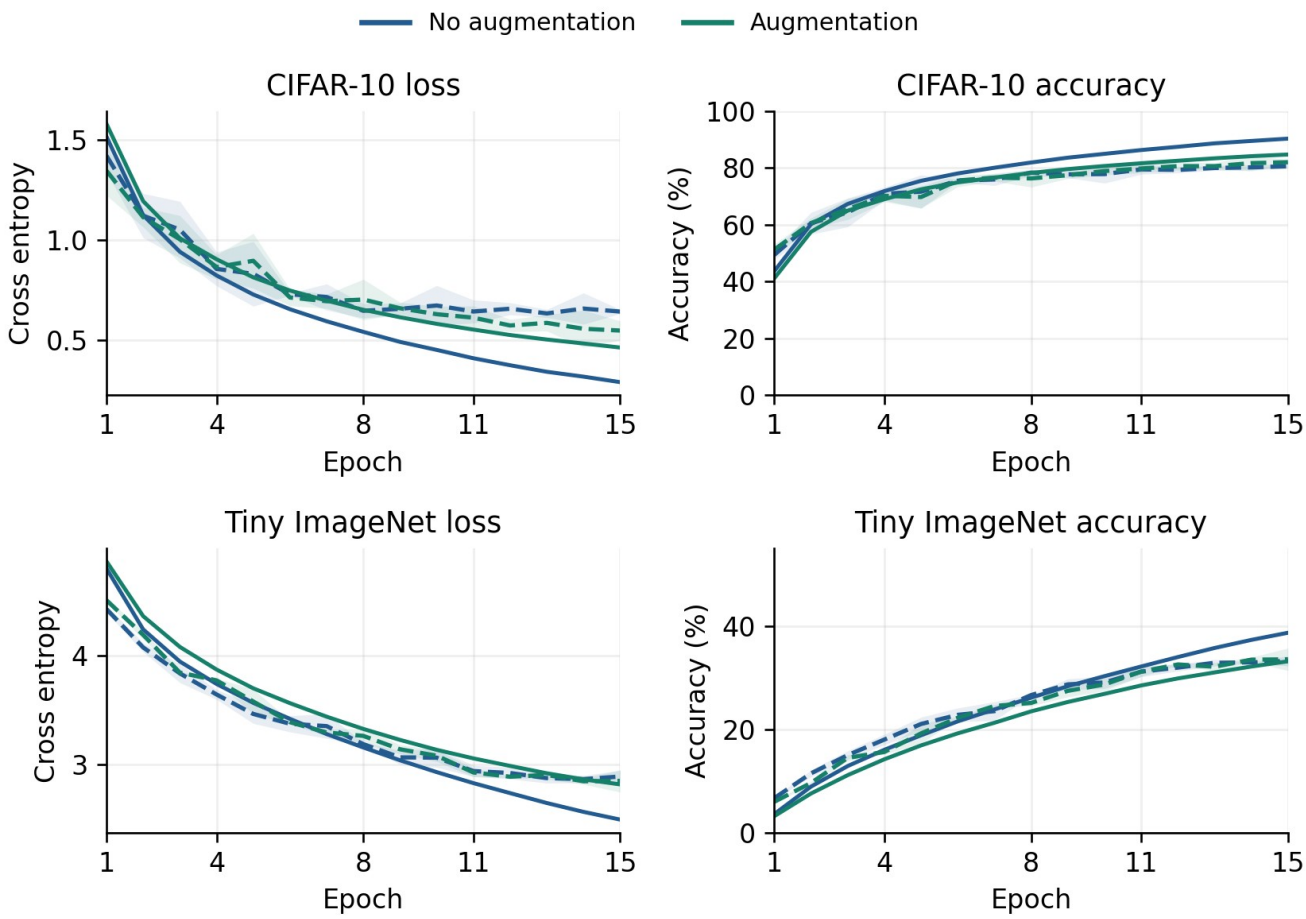


Figure 2 Matched 15-epoch BN and dropout runs, with and without augmentation. Solid training and dashed validation lines are three-seed means; validation shading is ±1 sample SD.

Augmentation reduced the mean clean training-validation gap from 10.27 to 4.04 points on CIFAR-10 and from 10.82 to 4.64 points on Tiny ImageNet. The answer to whether the gap contracted is yes for both datasets. CIFAR-10 mean validation accuracy increased by 1.43 points and Macro F1 by 1.39 points. Tiny ImageNet gains were only 0.30 and 0.22 points respectively, which are small relative to the observed variation.

The pipeline was Resize(64,64), reflected padding and RandomCrop(64), RandomHorizontalFlip(p=0.5), ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), ToTensor and dataset-specific normalization. These transformations preserve the intended class while varying position, orientation and appearance. The lower clean training accuracy and smaller gap are consistent with reduced memorization of the original training views.

The torchvision image transforms ran on CPU PIL images inside the data-loading workers before batches moved to CUDA. Validation and clean-training evaluation used deterministic resize, tensor conversion and normalization. The separate transform diagnostic demonstrates CUDA tensor transforms; it does not mean the training augmentation pipeline ran entirely on the GPU.

8 Learning rate exploration and dataset difficulty

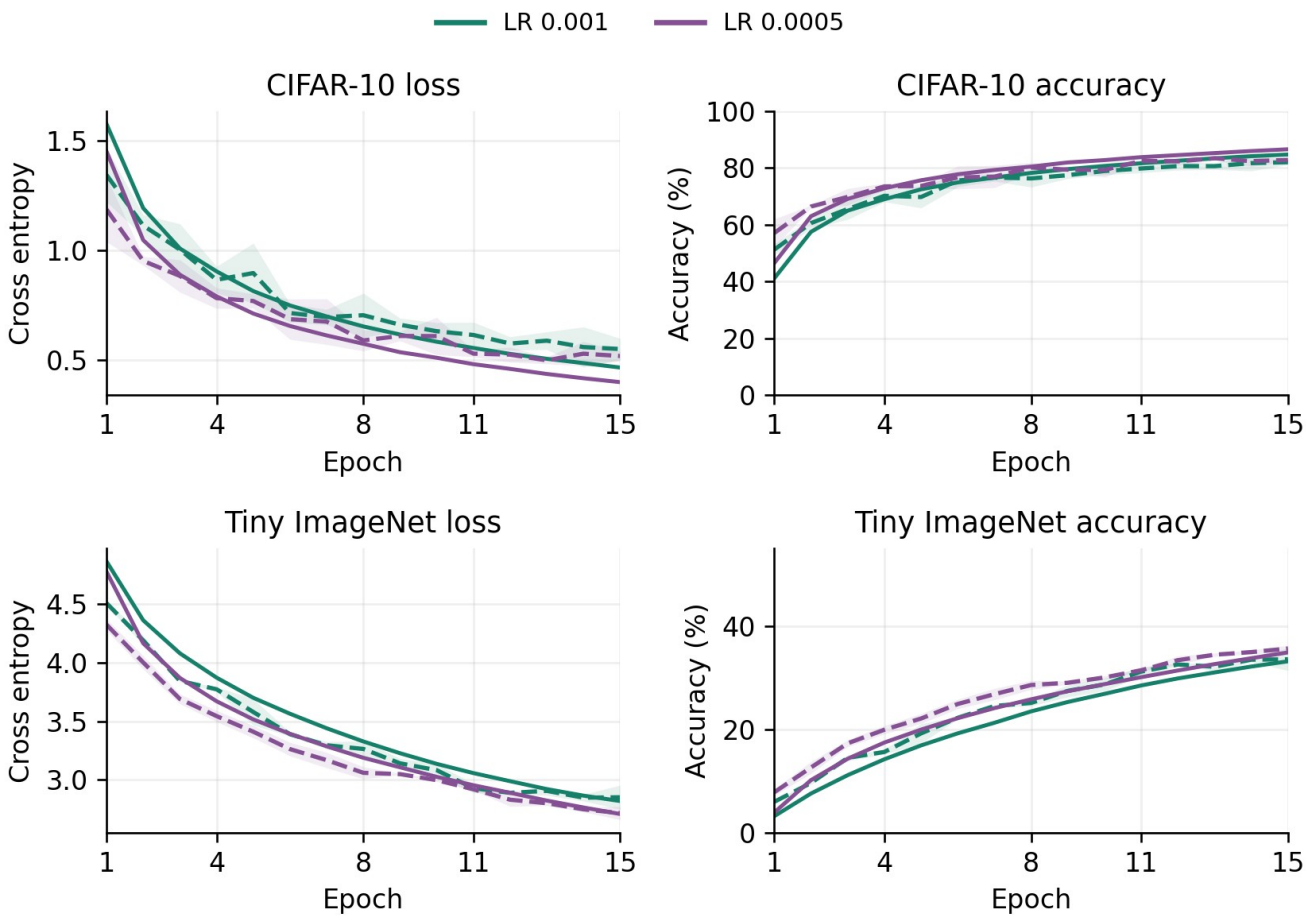


Figure 3 Custom learning-rate ablation, with BN, dropout and augmentation fixed for 15 epochs. Solid training and dashed validation lines are three-seed means; validation shading is ±1 sample SD.

Reducing the learning rate from 0.001 to 0.0005 increased mean validation accuracy by 0.81 points on CIFAR-10 and 2.09 points on Tiny ImageNet. This was the strongest tested final configuration on both datasets. Its gap was larger than that of the augmented 0.001 reference, by 0.77 and 1.37 points. A smaller gap alone therefore does not identify the most accurate model.

Tiny ImageNet has 200 classes and only 500 training examples per class, compared with 10 classes and 5,000 examples per class in CIFAR-10. Its lower scores across the ladder are consistent with the more demanding classification task. The best mean accuracy, 35.62%, is clearly above the 1/200 = 0.5% random-guess baseline. Its lower Macro F1, 34.26%, also shows that correct predictions are not distributed equally well across all classes despite balanced class counts.

Table 7 Tiny ImageNet seed sensitivity final accuracy in percent

Configuration	Seed 42	Seed 123	Seed 999
AlexNet LRN	0.50	15.57	15.35
Zoo AlexNet	0.50	11.58	12.79

Both seed-42 runs stayed near 0.5% accuracy, 0.004975% Macro F1 and cross entropy ln(200) ≈ 5.298. This is consistent with an uninformative predictor. The logs alone do not identify the cause or prove a specific prediction collapse. These runs remain in every aggregate. Zoo's rapid downsampling and large classifier may be poor fits for this input and budget, but they do not by themselves explain the failed seed.

9 Exploratory data analysis

The EDA inspected the training splits before standardization. Both datasets were perfectly balanced by class. The larger Tiny ImageNet task has twice as many training images in total, but one tenth as many examples per class.

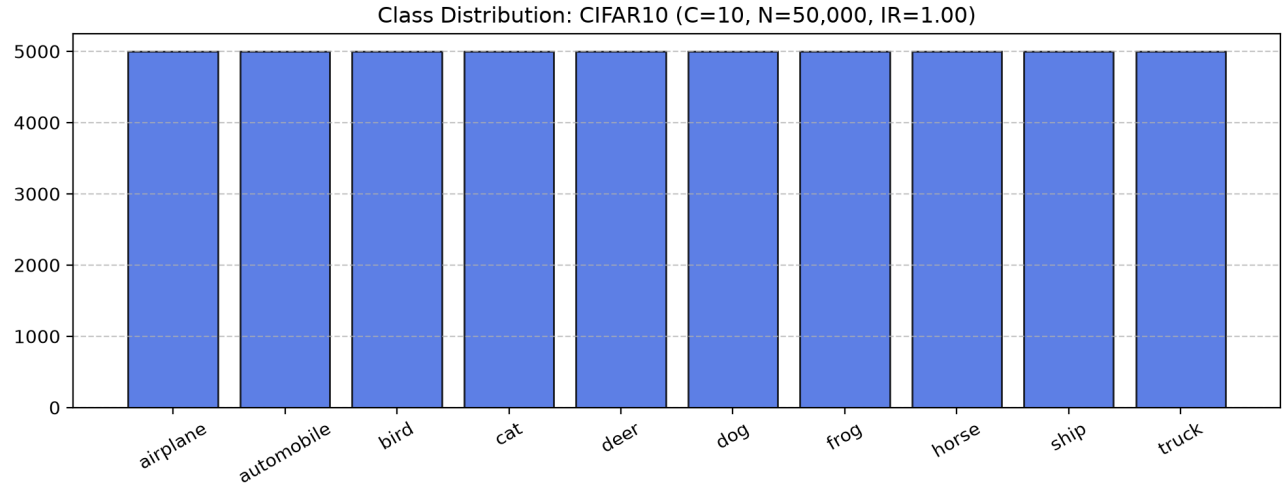


Figure 4 CIFAR-10 training class counts from utils/eda.py. Each of the ten classes has 5,000 images.

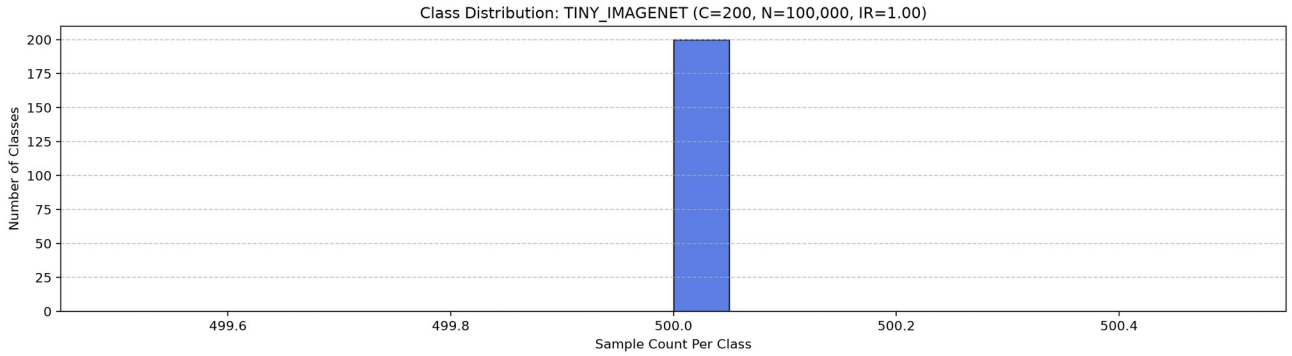


Figure 5 Tiny ImageNet histogram of per-class training counts from utils/eda.py. All 200 classes have 500 images.

Statistic	CIFAR-10	Tiny ImageNet
Training samples / classes	50,000 / 10	100,000 / 200
Samples per class	5,000	500
Max/min count ratio	1.00	1.00
Shannon entropy	3.3219 bits	7.6439 bits
Entropy / maximum	1.0000	1.0000
Gini impurity	0.9000	0.9950

Balanced counts remove a majority-class advantage, but they do not ensure equal per-class precision or recall. We therefore report both accuracy and Macro F1, as required by the lab. EDA completed without loading failures or nonfinite tensors. The threshold checks found no blank frames, with 1 dark and 4 bright CIFAR-10 images and 52 dark and 30 bright Tiny ImageNet images; these threshold counts alone do not establish that an image is invalid.

9 Exploratory data analysis continued

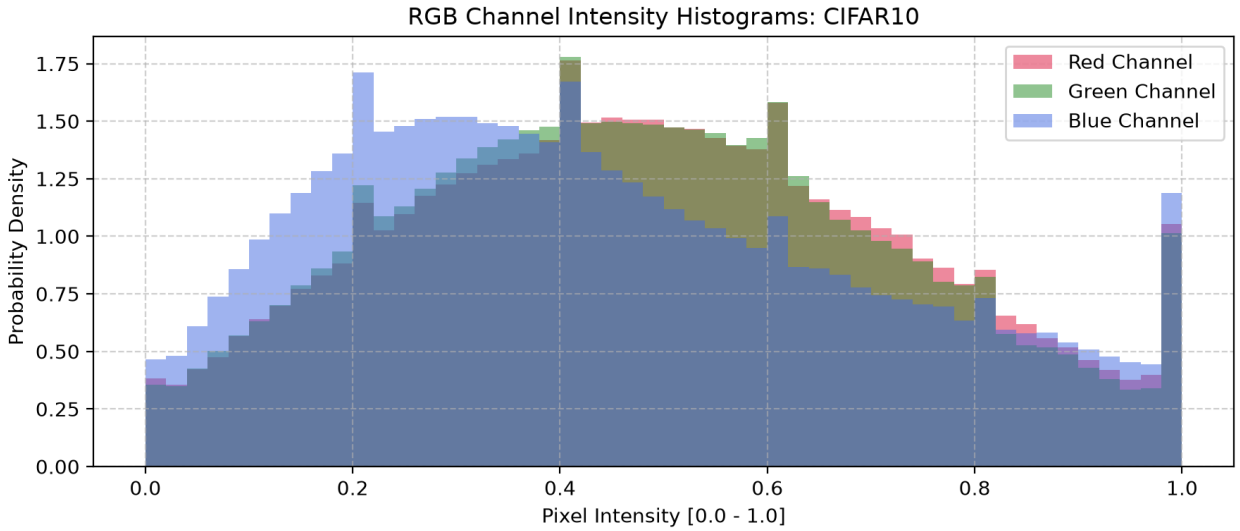


Figure 6 CIFAR-10 RGB feature distributions from utils/eda.py after resize and conversion to [0,1].

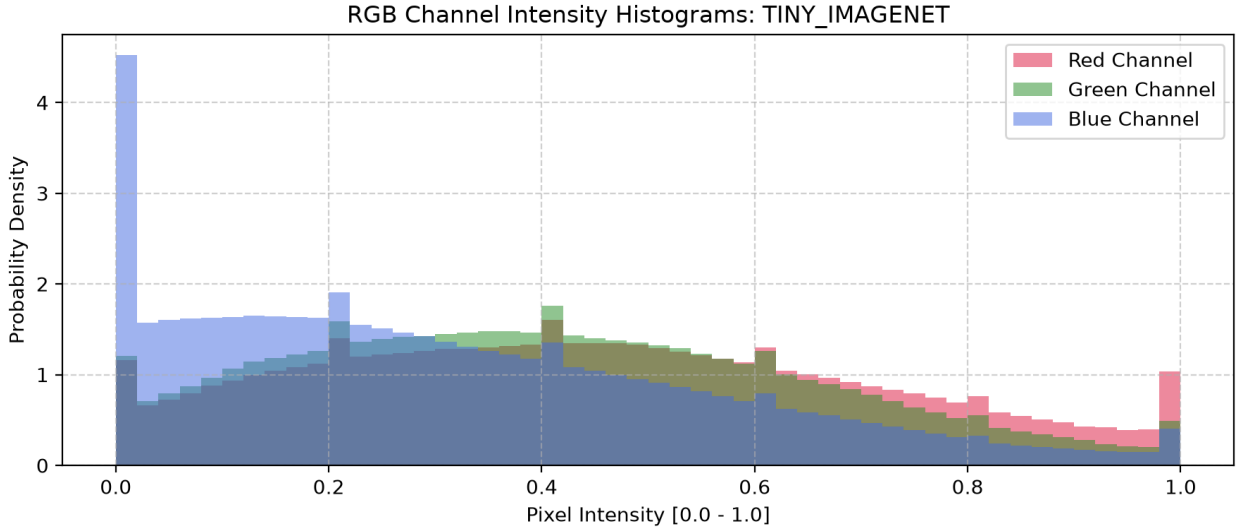


Figure 7 Tiny ImageNet RGB feature distributions from utils/eda.py after conversion to [0,1].

Training pixel statistic	CIFAR-10 R G B	Tiny ImageNet R G B
Measured mean at 64 × 64	0.4923 0.4831 0.4474	0.4802 0.4481 0.3975
Measured standard deviation	0.2404 0.2370 0.2557	0.2764 0.2689 0.2816
Training normalization mean	0.4914 0.4822 0.4465	0.4802 0.4481 0.3975
Training normalization std	0.2470 0.2435 0.2616	0.2764 0.2689 0.2821

Dataset-wide means and SDs use every training image after resize. Training retained the fixed constants in utils/transforms.py. These differ slightly from the EDA values, especially the CIFAR-10 SDs after interpolation to 64 × 64; the constants were not changed after training.

Histograms, percentiles and correlations use the first 2,000 loaded images. Tiny ImageNet's sorted ImageFolder includes only the first four class directories in this subset, so its histograms illustrate those classes. The means and SDs above use all training images. The saved plots also include sample grids.

10 Transform diagnostic and interpretation

The following excerpt records the object types, data layout and memory locations printed by demo_transforms.py. HWC is height × width × channels; CHW is channels × height × width. The PyTorch output is standardized and is no longer restricted to [0,1].

```
[1] OpenCV (cv2) Output:
    Object Type     : <class 'numpy.ndarray'>
    dtype           : float32
    Shape           : (64, 64, 3)  [HWC]
    Value Range     : 0.0000 to 1.0000
    Color Space     : Converted BGR -> RGB
    Memory Location : Host System RAM (CPU)

[2] NumPy Output:
    Object Type     : <class 'numpy.ndarray'>
    dtype           : float32
    Shape           : (64, 64, 3)  [HWC]
    Value Range     : 0.0000 to 1.0000
    Memory Location : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    Object Type     : <class 'torch.Tensor'>
    dtype           : torch.float32
    Shape           : torch.Size([3, 64, 64])  [CHW]
    Value Range     : -1.9895 to 2.1265
    Memory Location : cuda:0 (NVIDIA GPU VRAM)
```

Table 8 Printed diagnostic timings for 1,000 repeated small-image operations

Implementation	Total time ms	Images per second
OpenCV CPU	9.52	105,052
NumPy CPU	29.77	33,588
PyTorch CUDA tensor	339.79	2,943

The diagnostic's GPU loop was slower in this run. It repeatedly transforms a single 64 × 64 tensor, so per-operation overhead can dominate. The timed pipelines also perform different work: OpenCV and NumPy flip and scale to [0,1], whereas PyTorch flips and applies channel standardization. Despite the printed benchmark heading, resize is not part of these timed loops.

The CUDA input was copied to the GPU before timing and reused; the timed loop therefore excludes that initial transfer. CPU-to-GPU transfer was not measured for the CPU paths. These results demonstrate types, layout and residency, but do not establish an equivalent-work comparison or an end-to-end training speedup. Batched, equivalent pipelines would be needed for that comparison.

11 Architecture benchmark output and reproducibility

These tables reproduce python compare_models.py output on the RTX 3050 Laptop GPU: CUDA, batch size 1, 10 warmup forwards and 50 timed repetitions. M denotes millions; throughput is images per second.

Table 9a Terminal benchmark output CIFAR-10

Model	Params M	MACs M	FLOPs M	Latency ms	Images/s
Base LeNet	0.338486	3.586	7.172	0.4557	2194.4
Modern LeNet	0.446006	4.593	9.185	0.5250	1904.9
AlexNet none	6.608842	218.348	436.695	1.1260	888.1
AlexNet LRN	6.608842	218.348	436.695	1.4956	668.6
AlexNet BN	6.611402	218.983	437.966	1.1575	863.9
AlexNet BN + aug	6.611402	218.983	437.966	1.1855	843.5
Zoo AlexNet	57.044810	94.099	188.199	2.2456	445.3

Table 9b Terminal benchmark output Tiny ImageNet

Model	Params M	MACs M	FLOPs M	Latency ms	Images/s
Base LeNet	0.354636	3.602	7.204	0.4038	2476.2
Modern LeNet	0.462156	4.608	9.217	0.4368	2289.4
AlexNet none	6.803592	218.542	437.084	1.1585	863.2
AlexNet LRN	6.803592	218.542	437.084	1.4853	673.3
AlexNet BN	6.806152	219.178	438.355	1.1792	848.0
AlexNet BN + aug	6.806152	219.178	438.355	1.3020	768.0
Zoo AlexNet	57.823240	94.878	189.755	2.2994	434.9

Records and limitations

commands.txt records the training invocations. Each results JSON retains settings, seed, split sizes, history and final metrics. Checkpoints store the best validation-accuracy epoch; report tables use the final epoch. Figures 1-3 average all three seeds.

The short, unequal ladder budgets limit conclusions about eventual convergence. CIFAR-10 validation reuses its official test split, and learning-rate selection used the same validation sets. The selected score is therefore not an independent test estimate. Three seeds and variable system timing also limit comparisons.

References

[1] ELEC 475, Lab 2 Post-Lab Manual, Sections 3-9.

[2] ELEC 475, Lab 2 In-Lab Manual, LeNet architecture specifications.

[3] Lab 2 training JSONs, profile JSONs, transform and EDA logs, and accompanying source files.

[4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, PMLR 37, pp. 448-456, 2015. https://proceedings.mlr.press/v37/ioffe15.html

[5] S. Santurkar, D. Tsipras, A. Ilyas and A. Madry, “How Does Batch Normalization Help Optimization?” NeurIPS 31, 2018. https://arxiv.org/abs/1805.11604